



S-Move Dealer

Scale & Performance

Capacity math · when to change what

Covers the whole load — listing images (up to 5/item), listing views, business-page views, Votes and Hearts. Shows what the current stack handles comfortably, where it strains, and exactly which fix to reach for at which point.

Two headline findings

1. Tokens / Hearts / views rows are the cheap part — comfortable even at full saturation.
2. Images dominate — storage and especially bandwidth. Resize-on-upload + a CDN are the biggest single savings.

Prepared	3 July 2026	Stack	Node/Railway · Supabase Postgres + Storage
Companion to	Sponsored Sidebar / Votes / Hearts spec	Basis	40 customers per business

1 Assumptions & scale tiers

Every number is driven by these knobs — change one and the thresholds move. The math is rounded and slightly pessimistic; the goal is the *trigger points*, not row-exact totals.

Knob	Value	Note
Customers per business	40	your estimate
Items per business (avg)	30	real range ~10–100; scales linearly
Images per item	5 (max)	~4 likely average; 5 gives headroom
Original size (after on-upload resize)	~300 KB	if NOT resized, assume ~3 MB — see §5
Thumbnail size	~40 KB	480px WebP (generated today)
Business-page views / customer / mo	20	feeds sidebar ranking
Listing views / customer / mo	50	feeds marketplace ranking
Event row size (with indexes)	~200 bytes	id, fk, session, timestamp

The four scale tiers

Tier	Businesses	Customers	Items	Image files (×5)
Launch	50	2,000	1,500	7,500
Early	500	20,000	15,000	75,000
Growth	2,000	80,000	60,000	300,000
Saturation	8,000	320,000	240,000	1,200,000

2 Math A — database rows (tokens, hearts, views)

Event-row generation per month and year:

Tier	Votes/mo	Hearts/mo	Biz views/mo	Listing views/mo	Total /mo	Total /yr
Launch	2k	10k	40k	100k	~152k	~1.8M
Early	20k	100k	400k	1.0M	~1.5M	~18M
Growth	80k	400k	1.6M	4.0M	~6.1M	~73M
Saturation	320k	1.6M	6.4M	16M	~24M	~292M

Row storage (events × ~200 bytes):

Tier	Rows/yr	Storage/yr	3-yr (no pruning)
Launch	1.8M	~0.4 GB	~1 GB
Early	18M	~3.6 GB	~11 GB
Growth	73M	~15 GB	~44 GB
Saturation	292M	~58 GB	~175 GB

Verdict. A single well-indexed Postgres table is comfortable into the **hundreds of millions of rows**.

Launch→Growth need **zero special work** (Votes/Hearts are a rounding error; views are the bulk but modest). Only at **Saturation** do the view tables (~270M rows/yr combined) get large enough that rollups + partitioning + pruning (§7) earn their place — to keep queries fast and storage bounded, not to survive.

3 Math B — images (the real cost driver)

3a Storage

At ~340 KB per image slot (300 KB resized original + 40 KB thumbnail):

Tier	Image files	Storage @340 KB	If originals un-resized (~3 MB)
Launch	7,500	~2.6 GB	~23 GB
Early	75,000	~26 GB	~230 GB
Growth	300,000	~102 GB	~0.9 TB
Saturation	1,200,000	~408 GB	~3.6 TB

The right-hand column is why **resize-on-upload is the single biggest lever** — a ~10× difference in storage, and larger still in bandwidth.

3b Bandwidth (egress) — the one that bites

Images are re-served on every page load, so egress scales with **traffic**, not catalogue size. Browse page ~30 thumbnails ≈ 1.2 MB; item page (lazy-loaded) ~0.6–1.5 MB. At Saturation (~22M page views/mo) at ~1 MB images/page:

Monthly image egress	
No CDN (every hit from Supabase Storage)	~22 TB / mo
With CDN (~90%+ cache hit)	~2 TB / mo from origin

Verdict. Images are fine at Launch/Early on the default setup, but **resize-on-upload + CDN + lazy-loading are foundational** — cheap to add, they save the most later, and they make image loads fast for users immediately.

4 Comfort summary — traffic-light by tier

Tier	DB rows	Images (resized)	Egress	Overall
Launch (50 / 2k)	Trivial	~3 GB	Low	Nothing to do
Early (500 / 20k)	Comfortable	~26 GB	Rising	Add CDN + resize
Growth (2k / 80k)	Large view tables	~100 GB	Meaningful	Core upgrades \$7.1–4
Saturation (8k / 320k)	Needs rollups	~400 GB	CDN essential	Full kit \$7.1–7

Trigger metrics (watch these, not just business counts)

- **Recompute job runtime > ~60s** → set-based recompute + worker split (\$7.2–3)
- **Any hot event table > ~50M rows** → daily rollups + monthly partitioning + prune >90 days (\$7.4)
- **Supabase CPU sustained > ~70%, or slow browse/sidebar queries** → read replica / vertical scale (\$7.5–6)
- **DB connections near the pool ceiling** → Supabase pooler (\$7.3) — required *before* web replicas
- **Egress cost noticeable, or image pages feel slow** → CDN + resize (\$7.1) — do early regardless

5 The one thing to verify now

Are uploaded originals resized, or stored full-size?

Thumbnails (480px WebP) are already generated, but if originals are stored as raw phone photos (~3 MB), storage and egress follow the scary columns above. Capping originals to ~1600px at ~80% quality (~300 KB) on upload is a small change with a ~10× payoff and no visible quality loss. Confirm and, if needed, fix this before anything else — cheapest, highest-impact item in the document.

6 What breaks first if you do nothing

In the order you'd actually hit them:

- **Image egress cost / slow image pages** — climbs with traffic; earliest real pain. → CDN + resize.
- **In-process recompute multi-runs the moment you add web replicas** — `runEngagementScores` is a `setInterval` on the web process; N replicas = N concurrent recomputes. → worker/cron split (correctness, not just speed).
- **Per-item recompute loop slows** — one UPDATE per item, serial; drags at hundreds of thousands of items (low-thousands of businesses). → set-based update.
- **Hot view tables get big** — a year-plus into Growth/Saturation. → rollups + partitioning + pruning.

Nothing here breaks at launch or early — these are all “on the way up” items.

7 Solutions — what, when, how

Ordered by when you'd reach for them.

1 Image optimisation + CDN

WHEN: now / Early

What: Resize originals on upload; keep WebP thumbnails; lazy-load; serve all images through a CDN. Cheapest change, biggest payoff; images dominate storage and egress.

How:

- On upload, sharp resize originals to ~1600px, quality ~80 (thumbnails already exist).
- `loading="lazy"` on grids; load full image only on the item page.
- Put **Cloudflare** in front of Supabase Storage, or move objects to **Cloudflare R2** (zero egress); set long immutable `Cache-Control` (URLs are content-addressed).

2 Materialise sidebar counters + set-based recompute

WHEN: Growth (~1–2k businesses, or recompute > 60s)

What: Precompute counters onto the seller row on a schedule; replace the per-item update loop with one set-based statement. Stops the hot path counting event tables per card, and stops the recompute timing out.

How:

- Scheduled job runs one `UPDATE ... FROM (CTE)` instead of a per-row loop; store `views_30d`, `hearts_28d`, `sidebar_score` on the seller row.
- Sidebar read becomes a single indexed `ORDER BY sidebar_score` on stored columns.
- Indexes: `business_views(seller_id, created_at)`, `hearts_given(seller_id, created_at)`, `hearts_given(customer_id, created_at)`, `votes(customer_id, month)`.

3 Railway worker/cron split + Supabase pooler

WHEN: Growth (before any web replica)

What: Move recompute, AI tips and scraping into a dedicated worker on a cron; route app→DB through the pooler. Required before horizontal scaling — the in-process job would multi-run across replicas, and more replicas mean more DB connections.

How:

- New Railway service (same repo, different start command) runs the scheduled jobs; remove the in-process `setInterval` from the web process.
- Enable Supabase **transaction-mode pooler (pgBouncer)**; point the app connection string at the pooler.

4 Daily rollups + partitioning + pruning

WHEN: when a hot event table > ~50M rows

What: Small daily summary table; partition view tables by month; prune raw events past ~90 days. Rolling windows read ~28 tiny rows per seller instead of scanning hundreds of millions; raw tables stay permanently bounded.

How:

- Nightly job writes yesterday's per-seller/per-day counts into `daily_stats(seller_id, day, views, hearts)`; rolling 28-day windows sum from there.
- Convert view tables to Postgres **declarative partitioning by month**; drop/detach partitions past the retention window (lifetime heart totals live on the seller row, so all-time never needs the raw log).
- Dedup views by `session_id` at write time (one per session per entity per few hours) — cuts volume and kills refresh-spam gaming.

5 Read replica + web replicas

WHEN: five-figure concurrent-capable traffic

What: Add a read replica for the read-heavy browse/sidebar path; run multiple web instances behind Railway's load balancer. Reads vastly outnumber writes, so replicas absorb concurrency.

How:

- Enable a Supabase **read replica**; route read-only queries (browse, sidebar, item pages) to it, writes to the primary.
- Scale the Railway web service to N replicas — safe now the background job is separate and connections are pooled.

6 Vertical scale the Postgres primary

WHEN: sustained high CPU toward Saturation

What: Larger Supabase compute for the primary — simplest lever once rollups/partitioning are in and reads are offloaded; buys large headroom with no architectural change.

How:

- Upgrade the Supabase compute add-on for the primary; re-check slow-query logs after.

7 Distributed Postgres (Citus) / app sharding

WHEN: only if a single primary is truly outgrown — likely never

What: Shard Postgres horizontally within Postgres (Citus keeps joins/transactions). Reserved for tens of billions of rows / write saturation a single primary can't take — the numbers say you don't reach it at full regional saturation.

How:

- Shard on a key that keeps related data together — **not** by region (customers/hearts/votes cross regions; see the sidebar spec's rejected-alternatives note). Almost certainly unnecessary; prefer §7.4–6 first.

8 Recommended sequence

When	Do
Now / Launch	Verify + fix resize-on-upload ; add CDN + lazy-load (§7.1). Ship the feature set. Nothing else.
Growth (~1–2k businesses)	Worker/cron split + pooler (§7.3), set-based recompute + materialised counters (§7.2). Add indexes.
View table > ~50M rows	Daily rollups + monthly partitioning + prune >90d + view-dedup (§7.4).
Concurrency climbs	Read replica + web replicas (§7.5).
Toward saturation	Vertical-scale the primary (§7.6). Distribution (§7.7) only if truly outgrown — not expected.

9 TL;DR

- **The design scales to full regional saturation** (8k businesses / 320k customers). Tokens/hearts/views are cheap; only the view tables get large, handled by standard rollup/partition/prune patterns.
- **Images, not rows, are the cost story** — resize-on-upload and a CDN are foundational and should be done early; biggest single saving.
- **Nothing is needed at launch or early.** The first real work (worker split + pooler + set-based recompute) lands around the low thousands of businesses, driven by recompute runtime, not a subscriber count.
- **Watch the trigger metrics (§4), not the business count.** They tell you precisely when to switch each item on.

- **Don't shard by region** — a single primary with partitioning, a read replica, and a CDN carries you the whole way.

Cheap now, cheap later — because the order is known.

The architecture already fits every upgrade in this document without a rewrite. You turn each one on when its trigger metric fires — not before, not in a panic.